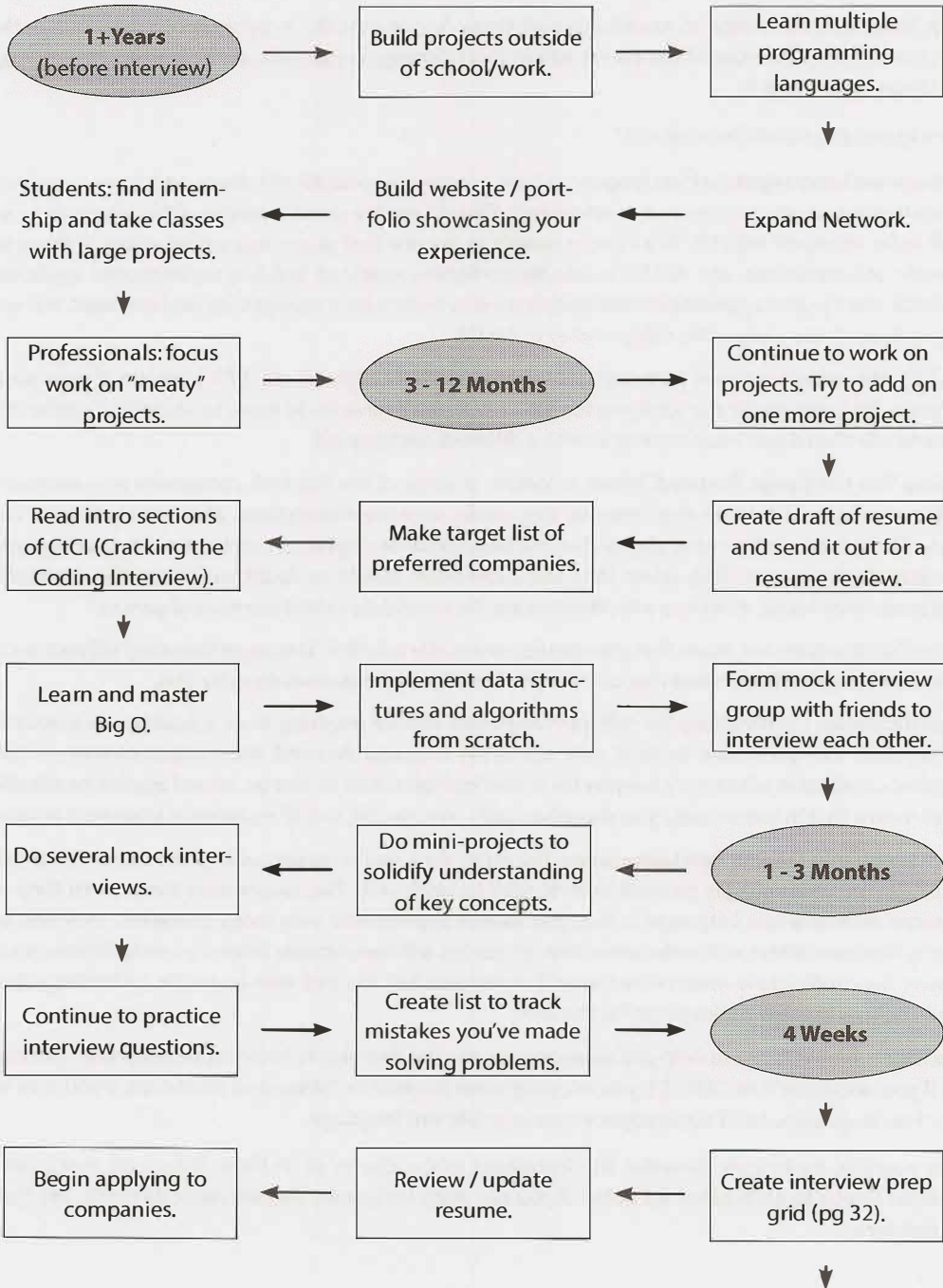
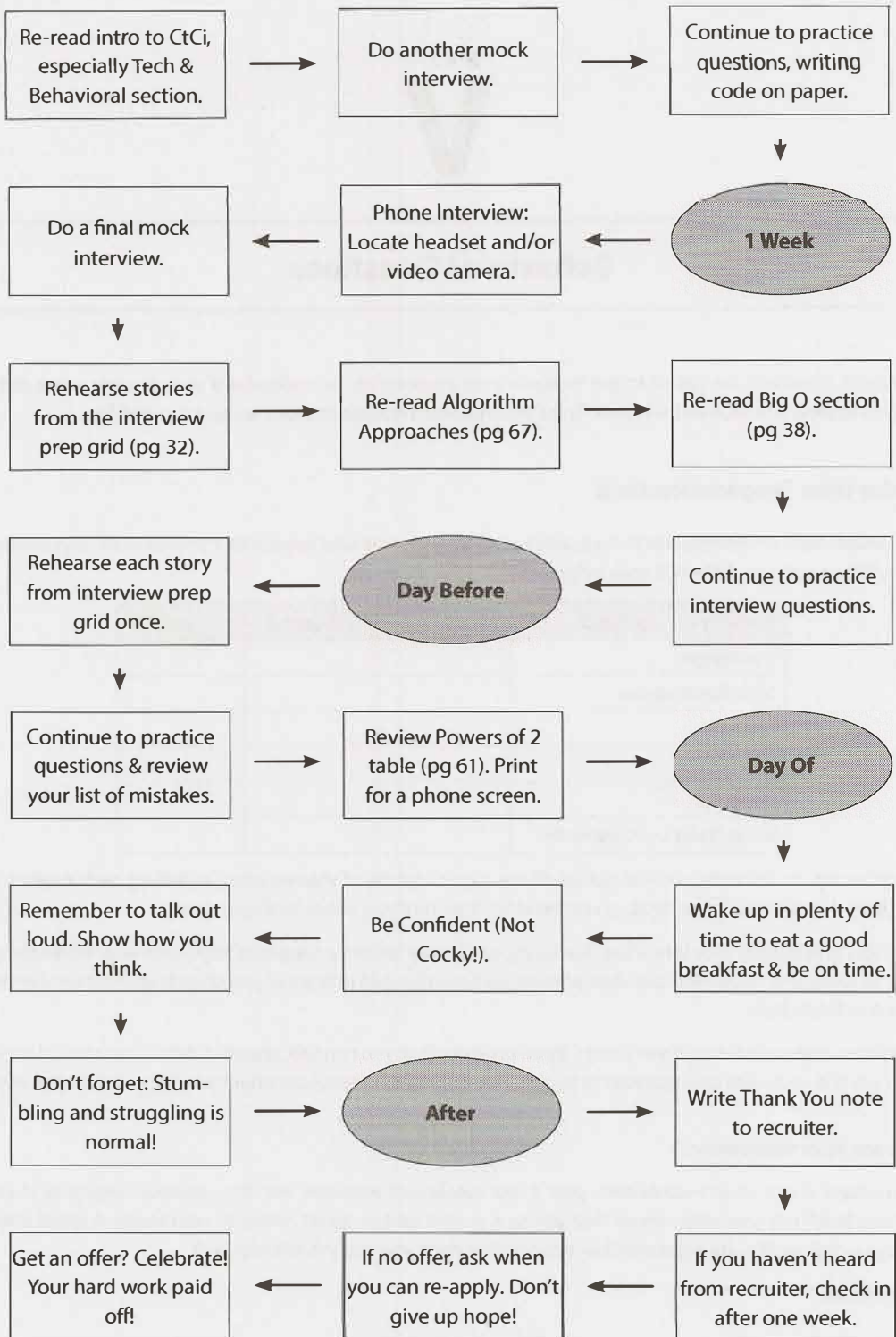


► Preparation Map

The following map should give you an idea of how to tackle the interview preparation process. One of the key takeaways here is that it's not just about interview questions. Do projects and write code, too!





V

Behavioral Questions

Behavioral questions are asked to get to know your personality, to understand your resume more deeply, and just to ease you into an interview. They are important questions and can be prepared for.

► Interview Preparation Grid

Go through each of the projects or components of your resume and ensure that you can talk about them in detail. Filling out a grid like this may help:

Common Questions	Project 1	Project 2	Project 3
Challenges			
Mistakes/Failures			
Enjoyed			
Leadership			
Conflicts			
What You'd Do Differently			

Along the top, as columns, you should list all the major aspects of your resume, including each project, job, or activity. Along the side, as rows, you should list the common behavioral questions.

Study this grid before your interview. Reducing each story to just a couple of keywords may make the grid easier to study and recall. You can also more easily have this grid in front of you during an interview without it being a distraction.

In addition, ensure that you have one to three projects that you can talk about in detail. You should be able to discuss the technical components in depth. These should be projects where you played a central role.

What are your weaknesses?

When asked about your weaknesses, give a real weakness! Answers like “My greatest weakness is that I work too hard” tell your interviewer that you’re arrogant and/or won’t admit to your faults. A good answer conveys a real, legitimate weakness but emphasizes how you work to overcome it.

For example:

“Sometimes, I don’t have a very good attention to detail. While that’s good because it lets me execute quickly, it also means that I sometimes make careless mistakes. Because of that, I make sure to always have someone else double check my work.”

What questions should you ask the interviewer?

Most interviewers will give you a chance to ask them questions. The quality of your questions will be a factor, whether subconsciously or consciously, in their decisions. Walk into the interview with some questions in mind.

You can think about three general types of questions.

Genuine Questions

These are the questions you actually want to know the answers to. Here are a few ideas of questions that are valuable to many candidates:

1. "What is the ratio of testers to developers to program managers? What is the interaction like? How does project planning happen on the team?"
2. "What brought you to this company? What has been most challenging for you?"

These questions will give you a good feel for what the day-to-day life is like at the company.

Insightful Questions

These questions demonstrate your knowledge or understanding of technology.

1. "I noticed that you use technology X. How do you handle problem Y?"
2. "Why did the product choose to use the X protocol over the Y protocol? I know it has benefits like A, B, C, but many companies choose not to use it because of issue D."

Asking such questions will typically require advance research about the company.

Passion Questions

These questions are designed to demonstrate your passion for technology. They show that you're interested in learning and will be a strong contributor to the company.

1. "I'm very interested in scalability, and I'd love to learn more about it. What opportunities are there at this company to learn about this?"
2. "I'm not familiar with technology X, but it sounds like a very interesting solution. Could you tell me a bit more about how it works?"

► Know Your Technical Projects

As part of your preparation, you should focus on two or three technical projects that you should deeply master. Select projects that ideally fit the following criteria:

- The project had challenging components (beyond just "learning a lot").
- You played a central role (ideally on the challenging components).
- You can talk at technical depth.

For those projects, and all your projects, be able to talk about the challenges, mistakes, technical decisions, choices of technologies (and tradeoffs of these), and the things you would do differently.

You can also think about follow-up questions, like how you would scale the application.

► Responding to Behavioral Questions

Behavioral questions allow your interviewer to get to know you and your prior experience better. Remember the following advice when responding to questions.

Be Specific, Not Arrogant

Arrogance is a red flag, but you still want to make yourself sound impressive. So how do you make yourself sound good without being arrogant? By being specific!

Specificity means giving just the facts and letting the interviewer derive an interpretation. For example, rather than saying that you “did all the hard parts,” you can instead describe the specific bits you did that were challenging.

Limit Details

When a candidate blabbers on about a problem, it’s hard for an interviewer who isn’t well versed in the subject or project to understand it.

Stay light on details and just state the key points. When possible, try to translate it or at least explain the impact. You can always offer the interviewer the opportunity to drill in further.

“By examining the most common user behavior and applying the Rabin-Karp algorithm, I designed a new algorithm to reduce search from $O(n)$ to $O(\log n)$ in 90% of cases. I can go into more details if you’d like.”

This demonstrates the key points while letting your interviewer ask for more details if he wants to.

Focus on Yourself, Not Your Team

Interviews are fundamentally an individual assessment. Unfortunately, when you listen to many candidates (especially those in leadership roles), their answers are about “we,” “us,” and “the team.” The interviewer walks away having little idea what the candidate’s actual impact was and might conclude that the candidate did little.

Pay attention to your answers. Listen for how much you say “we” versus “I.” Assume that every question is about your role, and speak to that.

Give Structured Answers

There are two common ways to think about structuring responses to a behavioral question: nugget first and S.A.R. These techniques can be used separately or together.

Nugget First

Nugget First means starting your response with a “nugget” that succinctly describes what your response will be about.

For example:

- Interviewer: “Tell me about a time you had to persuade a group of people to make a big change.”
- Candidate: “Sure, let me tell you about the time when I convinced my school to let undergraduates teach their own courses. Initially, my school had a rule where...”

This technique grabs your interviewer's attention and makes it very clear what your story will be about. It also helps you be more focused in your communication, since you've made it very clear to yourself what the gist of your response is.

S.A.R. (Situation, Action, Result)

The S.A.R. approach means that you start off outlining the situation, then explaining the actions you took, and lastly, describing the result.

Example: "Tell me about a challenging interaction with a teammate."

- **Situation:** On my operating systems project, I was assigned to work with three other people. While two were great, the third team member didn't contribute much. He stayed quiet during meetings, rarely chipped in during email discussions, and struggled to complete his components. This was an issue not only because it shifted more work onto us, but also because we didn't know if we could count on him.

- **Action:** I didn't want to write him off completely yet, so I tried to resolve the situation. I did three things. First, I wanted to understand why he was acting like this. Was it laziness? Was he busy with something else? I struck up a conversation with him and then asked him open-ended questions about how he felt it was going. Interestingly, basically out of nowhere, he said that he wanted to take on the writeup, which is one of the most time intensive parts. This showed me that it wasn't laziness; it was that he didn't feel like he was good enough to write code.

Second, now that I understand the cause, I tried to make it clear that he shouldn't fear messing up. I told him about some of the bigger mistakes that I made and admitted that I wasn't clear about a lot of parts of the project either.

Third and finally, I asked him to help me with breaking out some of the components of the project. We sat down together and designed a thorough spec for one of the big component, in much more detail than we had before. Once he could see all the pieces, it helped show him that the project wasn't as scary as he'd assumed.

- **Result:** With his confidence raised, he now offered to take on a bunch of the smaller coding work, and then eventually some of the biggest parts. He finished all his work on time, and he contributed more in discussions. We were happy to work with him on a future project.

The situation and the result should be succinct. Your interviewer generally does not need many details to understand what happened and, in fact, may be confused by them.

By using the S.A.R. model with clear situations, actions and results, the interviewer will be able to easily identify how you made an impact and why it mattered.

Consider putting your stories into the following grid:

	Nugget	Situation	Action(s)	Result	What It Says
Story 1			1. ... 2. ... 3. ...		
Story 2					

Explore the Action

In almost all cases, the "action" is the most important part of the story. Unfortunately, far too many people talk on and on about the situation, but then just breeze through the action.

Instead, dive into the action. Where possible, break down the action into multiple parts. For example: “I did three things. First, I...” This will encourage sufficient depth.

Think About What It Says

Re-read the story on page 35. What personality attributes has the candidate demonstrated?

- **Initiative/Leadership:** The candidate tried to resolve the situation by addressing it head-on.
- **Empathy:** The candidate tried to understand what was happening to the person. The candidate also showed empathy in knowing what would resolve the teammate’s insecurity.
- **Compassion:** Although the teammate was harming the team, the candidate wasn’t angry at the teammate. His empathy led him to compassion.
- **Humility:** The candidate was able to admit to his own flaws (not only to the teammate, but also to the interviewer).
- **Teamwork/Helpfulness:** The candidate worked with the teammate to break down the project into manageable chunks.

You should think about your stories from this perspective. Analyze the actions you took and how you reacted. What personality attributes does your reaction demonstrate?

In many cases, the answer is “none.” That usually means you need to rework how you communicate the story to make the attribute clearer. You don’t want to explicitly say, “I did X because I have empathy,” but you can go one step away from that. For example:

- **Less Clear Attribute:** “I called up the client and told him what happened.”
- **More Clear Attribute (Empathy and Courage):** “I made sure to call the client myself, because I knew that he would appreciate hearing it directly from me.”

If you still can’t make the personality attributes clear, then you might need to come up with a new story entirely.

► So, tell me about yourself...

Many interviewers kick off the session by asking you to tell them a bit about yourself, or asking you to walk through your resume. This is essentially a “pitch”. It’s your interviewer’s first impression of you, so you want to be sure to nail this.

Structure

A typical structure that works well for many people is essentially chronological, with the opening sentence describing their current job and the conclusion discussing their relevant and interesting hobbies outside of work (if any).

1. **Current Role [Headline Only]:** “I’m a software engineer at Microworks, where I’ve been leading the Android team for the last five years.”
2. **College:** My background is in computer science. I did my undergrad at Berkeley and spent a few summers working at startups, including one where I attempted to launch my own business.
3. **Post College & Onwards:** After college, I wanted to get some exposure to larger corporations so I joined Amazon as a developer. It was a great experience. I learned a ton about large system design and I got to really drive the launch of a key part of AWS. That actually showed me that I really wanted to be in a more

entrepreneurial environment.

4. **Current Role [Details]:** One of my old managers from Amazon recruited me out to join her startup, which was what brought me to Microworks. Here, I did the initial system architecture, which has scaled pretty well with our rapid growth. I then took an opportunity to lead the Android team. I do manage a team of three, but my role is primarily with technical leadership: architecture, coding, etc.
5. **Outside of Work:** Outside of work, I've been participating in some hackathons—mostly doing iOS development there as a way to learn it more deeply. I'm also active as a moderator on online forums around Android development.
6. **Wrap Up:** I'm looking now for something new, and your company caught my eye. I've always loved the connection with the user, and I really want to get back to a smaller environment too.

This structure works well for about 95% of candidates. For candidate with more experience, you might condense part of it. Ten years from now, the candidate's initial statements might become just: "After my CS degree from Berkeley, I spent a few years at Amazon and then joined a startup where I led the Android team."

Hobbies

Think carefully about your hobbies. You may or may not want to discuss them.

Often they're just fluff. If your hobby is just generic activities like skiing or playing with your dog, you can probably skip it.

Sometimes though, hobbies can be useful. This often happens when:

- The hobby is extremely unique (e.g., fire breathing). It may strike up a bit of a conversation and kick off the interview on a more amiable note.
- The hobby is technical. This not only boosts your actual skillset, but it also shows passion for technology.
- The hobby demonstrates a positive personality attribute. A hobby like "remodeling your house yourself" shows a drive to learn new things, take some risks, and get your hands dirty (literally and figuratively).

It would rarely hurt to mention hobbies, so when in doubt, you might as well.

Think about how to best frame your hobby though. Do you have any successes or specific work to show from it (e.g., landing a part in a play)? Is there a personality attribute this hobby demonstrates?

Sprinkle in Shows of Successes

In the above pitch, the candidate has casually dropped in some highlights of his background.

- He specifically mentioned that he was recruited out of Microworks by his old manager, which shows that he was successful at Amazon.
- He also mentions wanting to be in a smaller environment, which shows some element of culture fit (assuming this is a startup he's applying for).
- He mentions some successes he's had, such as launching a key part of AWS and architecting a scalable system.
- He mentions his hobbies, both of which show a drive to learn.

When you think about your pitch, think about what different aspects of your background say about you. Can you drop in shows of successes (awards, promotions, being recruited out by someone you worked with, launches, etc.)? What do you want to communicate about yourself?

VI

Big O

This is such an important concept that we are dedicating an entire (long!) chapter to it.

Big O time is the language and metric we use to describe the efficiency of algorithms. Not understanding it thoroughly can really hurt you in developing an algorithm. Not only might you be judged harshly for not really understanding big O, but you will also struggle to judge when your algorithm is getting faster or slower.

Master this concept.

► An Analogy

Imagine the following scenario: You've got a file on a hard drive and you need to send it to your friend who lives across the country. You need to get the file to your friend as fast as possible. How should you send it?

Most people's first thought would be email, FTP, or some other means of electronic transfer. That thought is reasonable, but only half correct.

If it's a small file, you're certainly right. It would take 5 - 10 hours to get to an airport, hop on a flight, and then deliver it to your friend.

But what if the file were really, really large? Is it possible that it's faster to physically deliver it via plane?

Yes, actually it is. A one-terabyte (1 TB) file could take more than a day to transfer electronically. It would be much faster to just fly it across the country. If your file is that urgent (and cost isn't an issue), you might just want to do that.

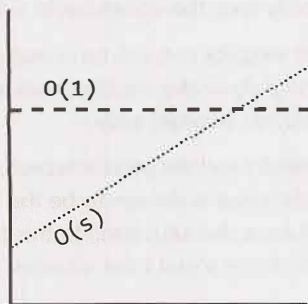
What if there were no flights, and instead you had to drive across the country? Even then, for a really huge file, it would be faster to drive.

► Time Complexity

This is what the concept of asymptotic runtime, or big O time, means. We could describe the data transfer "algorithm" runtime as:

- Electronic Transfer: $O(s)$, where s is the size of the file. This means that the time to transfer the file increases linearly with the size of the file. (Yes, this is a bit of a simplification, but that's okay for these purposes.)
- Airplane Transfer: $O(1)$ with respect to the size of the file. As the size of the file increases, it won't take any longer to get the file to your friend. The time is constant.

No matter how big the constant is and how slow the linear increase is, linear will at some point surpass constant.



There are many more runtimes than this. Some of the most common ones are $O(\log N)$, $O(N \log N)$, $O(N)$, $O(N^2)$ and $O(2^N)$. There's no fixed list of possible runtimes, though.

You can also have multiple variables in your runtime. For example, the time to paint a fence that's w meters wide and h meters high could be described as $O(wh)$. If you needed p layers of paint, then you could say that the time is $O(whp)$.

Big O, Big Theta, and Big Omega

If you've never covered big O in an academic setting, you can probably skip this subsection. It might confuse you more than it helps. This "FYI" is mostly here to clear up ambiguity in wording for people who have learned big O before, so that they don't say, "But I thought big O meant..."

Academics use big O, big Θ (theta), and big Ω (omega) to describe runtimes.

- **O (big O):** In academia, big O describes an upper bound on the time. An algorithm that prints all the values in an array could be described as $O(N)$, but it could also be described as $O(N^2)$, $O(N^3)$, or $O(2^N)$ (or many other big O times). The algorithm is at least as fast as each of these; therefore they are upper bounds on the runtime. This is similar to a less-than-or-equal-to relationship. If Bob is X years old (I'll assume no one lives past age 130), then you could say $X \leq 130$. It would also be correct to say that $X \leq 1,000$ or $X \leq 1,000,000$. It's technically true (although not terribly useful). Likewise, a simple algorithm to print the values in an array is $O(N)$ as well as $O(N^3)$ or any runtime bigger than $O(N)$.
- **Ω (big omega):** In academia, Ω is the equivalent concept but for lower bound. Printing the values in an array is $\Omega(N)$ as well as $\Omega(\log N)$ and $\Omega(1)$. After all, you know that it won't be *faster* than those runtimes.
- **Θ (big theta):** In academia, Θ means both O and Ω . That is, an algorithm is $\Theta(N)$ if it is both $O(N)$ and $\Omega(N)$. Θ gives a tight bound on runtime.

In industry (and therefore in interviews), people seem to have merged Θ and O together. Industry's meaning of big O is closer to what academics mean by Θ , in that it would be seen as incorrect to describe printing an array as $O(N^2)$. Industry would just say this is $O(N)$.

For this book, we will use big O in the way that industry tends to use it: By always trying to offer the tightest description of the runtime.

Best Case, Worst Case, and Expected Case

We can actually describe our runtime for an algorithm in three different ways.

Let's look at this from the perspective of quick sort. Quick sort picks a random element as a "pivot" and then swaps values in the array such that the elements less than pivot appear before elements greater than pivot. This gives a "partial sort." Then it recursively sorts the left and right sides using a similar process.

- **Best Case:** If all elements are equal, then quick sort will, on average, just traverse through the array once. This is $O(N)$. (This actually depends slightly on the implementation of quick sort. There are implementations, though, that will run very quickly on a sorted array.)
- **Worst Case:** What if we get really unlucky and the pivot is repeatedly the biggest element in the array? (Actually, this can easily happen. If the pivot is chosen to be the first element in the subarray and the array is sorted in reverse order, we'll have this situation.) In this case, our recursion doesn't divide the array in half and recurse on each half. It just shrinks the subarray by one element. This will degenerate to an $O(N^2)$ runtime.
- **Expected Case:** Usually, though, these wonderful or terrible situations won't happen. Sure, sometimes the pivot will be very low or very high, but it won't happen over and over again. We can expect a runtime of $O(N \log N)$.

We rarely ever discuss best case time complexity, because it's not a very useful concept. After all, we could take essentially any algorithm, special case some input, and then get an $O(1)$ time in the best case.

For many—probably most—algorithms, the worst case and the expected case are the same. Sometimes they're different, though, and we need to describe both of the runtimes.

What is the relationship between best/worst/expected case and big O/theta/omega?

It's easy for candidates to muddle these concepts (probably because both have some concepts of "higher", "lower" and "exactly right"), but there is no particular relationship between the concepts.

Best, worst, and expected cases describe the big O (or big theta) time for particular inputs or scenarios.

Big O, big omega, and big theta describe the upper, lower, and tight bounds for the runtime.

► Space Complexity

Time is not the only thing that matters in an algorithm. We might also care about the amount of memory—or space—required by an algorithm.

Space complexity is a parallel concept to time complexity. If we need to create an array of size n , this will require $O(n)$ space. If we need a two-dimensional array of size $n \times n$, this will require $O(n^2)$ space.

Stack space in recursive calls counts, too. For example, code like this would take $O(n)$ time and $O(n)$ space.

```
1  int sum(int n) { /* Ex 1.*/
2      if (n <= 0) {
3          return 0;
4      }
5      return n + sum(n-1);
6  }
```

Each call adds a level to the stack.

```
1  sum(4)
2      -> sum(3)
3          -> sum(2)
4              -> sum(1)
5                  -> sum(0)
```

Each of these calls is added to the call stack and takes up actual memory.

However, just because you have n calls total doesn't mean it takes $O(n)$ space. Consider the below function, which adds adjacent elements between 0 and n :

```
1 int pairSumSequence(int n) { /* Ex 2.*/
2     int sum = 0;
3     for (int i = 0; i < n; i++) {
4         sum += pairSum(i, i + 1);
5     }
6     return sum;
7 }
8
9 int pairSum(int a, int b) {
10     return a + b;
11 }
```

There will be roughly $O(n)$ calls to `pairSum`. However, those calls do not exist simultaneously on the call stack, so you only need $O(1)$ space.

► Drop the Constants

It is very possible for $O(N)$ code to run faster than $O(1)$ code for specific inputs. Big O just describes the rate of increase.

For this reason, we drop the constants in runtime. An algorithm that one might have described as $O(2N)$ is actually $O(N)$.

Many people resist doing this. They will see code that has two (non-nested) for loops and continue this $O(2N)$. They think they're being more "precise." They're not.

Consider the below code:

Min and Max 1

```
1 int min = Integer.MAX_VALUE;
2 int max = Integer.MIN_VALUE;
3 for (int x : array) {
4     if (x < min) min = x;
5     if (x > max) max = x;
6 }
```

Min and Max 2

```
1 int min = Integer.MAX_VALUE;
2 int max = Integer.MIN_VALUE;
3 for (int x : array) {
4     if (x < min) min = x;
5 }
6 for (int x : array) {
7     if (x > max) max = x;
8 }
```

Which one is faster? The first one does one for loop and the other one does two for loops. But then, the first solution has two lines of code per for loop rather than one.

If you're going to count the number of instructions, then you'd have to go to the assembly level and take into account that multiplication requires more instructions than addition, how the compiler would optimize something, and all sorts of other details.

This would be horrendously complicated, so don't even start going down this road. Big O allows us to express how the runtime scales. We just need to accept that it doesn't mean that $O(N)$ is always better than $O(N^2)$.

► Drop the Non-Dominant Terms

What do you do about an expression such as $O(N^2 + N)$? That second N isn't exactly a constant. But it's not especially important.

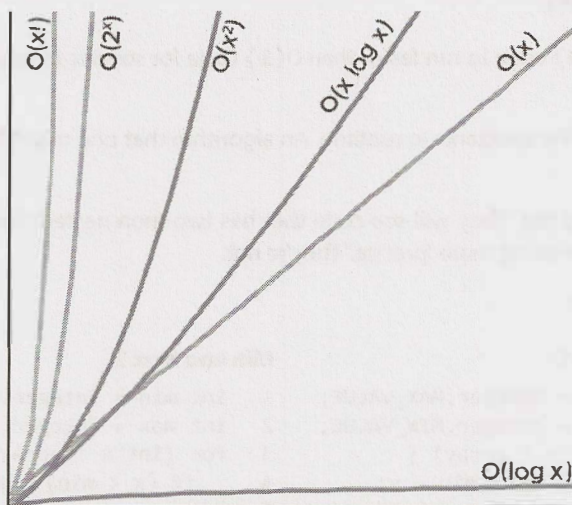
We already said that we drop constants. Therefore, $O(N^2 + N^2)$ would be $O(N^2)$. If we don't care about that latter N^2 term, why would we care about N ? We don't.

You should drop the non-dominant terms.

- $O(N^2 + N)$ becomes $O(N^2)$.
- $O(N + \log N)$ becomes $O(N)$.
- $O(5 \cdot 2^N + 1000N^{100})$ becomes $O(2^N)$.

We might still have a sum in a runtime. For example, the expression $O(B^2 + A)$ cannot be reduced (without some special knowledge of A and B).

The following graph depicts the rate of increase for some of the common big O times.



As you can see, $O(x^2)$ is much worse than $O(x)$, but it's not nearly as bad as $O(2^x)$ or $O(x!)$. There are lots of runtimes worse than $O(x!)$ too, such as $O(x^x)$ or $O(2^x * x!)$.

► Multi-Part Algorithms: Add vs. Multiply

Suppose you have an algorithm that has two steps. When do you multiply the runtimes and when do you add them?

This is a common source of confusion for candidates.

Add the Runtimes: $O(A + B)$

```

1  for (int a : arrA) {
2      print(a);
3  }
4
5  for (int b : arrB) {
6      print(b);
7  }

```

Multiply the Runtimes: $O(A*B)$

```

1  for (int a : arrA) {
2      for (int b : arrB) {
3          print(a + "," + b);
4      }
5  }

```

In the example on the left, we do A chunks of work then B chunks of work. Therefore, the total amount of work is $O(A + B)$.

In the example on the right, we do B chunks of work for each element in A. Therefore, the total amount of work is $O(A * B)$.

In other words:

- If your algorithm is in the form “do this, then, when you’re all done, do that” then you add the runtimes.
- If your algorithm is in the form “do this for each time you do that” then you multiply the runtimes.

It’s very easy to mess this up in an interview, so be careful.

► Amortized Time

An `ArrayList`, or a dynamically resizing array, allows you to have the benefits of an array while offering flexibility in size. You won’t run out of space in the `ArrayList` since its capacity will grow as you insert elements.

An `ArrayList` is implemented with an array. When the array hits capacity, the `ArrayList` class will create a new array with double the capacity and copy all the elements over to the new array.

How do you describe the runtime of insertion? This is a tricky question.

The array could be full. If the array contains N elements, then inserting a new element will take $O(N)$ time. You will have to create a new array of size $2N$ and then copy N elements over. This insertion will take $O(N)$ time.

However, we also know that this doesn’t happen very often. The vast majority of the time insertion will be in $O(1)$ time.

We need a concept that takes both into account. This is what amortized time does. It allows us to describe that, yes, this worst case happens every once in a while. But once it happens, it won’t happen again for so long that the cost is “amortized.”

In this case, what is the amortized time?

As we insert elements, we double the capacity when the size of the array is a power of 2. So after X elements, we double the capacity at array sizes 1, 2, 4, 8, 16, ..., X . That doubling takes, respectively, 1, 2, 4, 8, 16, 32, 64, ..., X copies.

What is the sum of $1 + 2 + 4 + 8 + 16 + \dots + X$? If you read this sum left to right, it starts with 1 and doubles until it gets to X . If you read right to left, it starts with X and halves until it gets to 1.

What then is the sum of $X + \frac{X}{2} + \frac{X}{4} + \frac{X}{8} + \dots + 1$? This is roughly $2X$.

Therefore, X insertions take $O(2X)$ time. The amortized time for each insertion is $O(1)$.

► Log N Runtimes

We commonly see $O(\log N)$ in runtimes. Where does this come from?

Let's look at binary search as an example. In binary search, we are looking for an example x in an N -element sorted array. We first compare x to the midpoint of the array. If $x == \text{middle}$, then we return. If $x < \text{middle}$, then we search on the left side of the array. If $x > \text{middle}$, then we search on the right side of the array.

```
search 9 within {1, 5, 8, 9, 11, 13, 15, 19, 21}
  compare 9 to 11 -> smaller.
    search 9 within {1, 5, 8, 9, 11}
      compare 9 to 8 -> bigger
        search 9 within {9, 11}
          compare 9 to 9
            return
```

We start off with an N -element array to search. Then, after a single step, we're down to $\frac{N}{2}$ elements. One more step, and we're down to $\frac{N}{4}$ elements. We stop when we either find the value or we're down to just one element.

The total runtime is then a matter of how many steps (dividing N by 2 each time) we can take until N becomes 1.

```
N = 16
N = 8      /* divide by 2 */
N = 4      /* divide by 2 */
N = 2      /* divide by 2 */
N = 1      /* divide by 2 */
```

We could look at this in reverse (going from 1 to 16 instead of 16 to 1). How many times we can multiply 1 by 2 until we get N ?

```
N = 1
N = 2      /* multiply by 2 */
N = 4      /* multiply by 2 */
N = 8      /* multiply by 2 */
N = 16     /* multiply by 2 */
```

What is k in the expression $2^k = N$? This is exactly what $\log$ expresses.

```
24 = 16 -> log216 = 4
log2N = k -> 2k = N
```

This is a good takeaway for you to have. When you see a problem where the number of elements in the problem space gets halved each time, that will likely be a $O(\log N)$ runtime.

This is the same reason why finding an element in a balanced binary search tree is $O(\log N)$. With each comparison, we go either left or right. Half the nodes are on each side, so we cut the problem space in half each time.

What's the base of the log? That's an excellent question! The short answer is that it doesn't matter for the purposes of big O. The longer explanation can be found at "Bases of Logs" on page 630.

► Recursive Runtimes

Here's a tricky one. What's the runtime of this code?

```
1 int f(int n) {
```

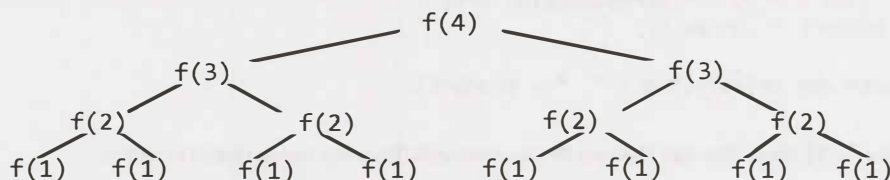
```

2   if (n <= 1) {
3       return 1;
4   }
5   return f(n - 1) + f(n - 1);
6   }

```

A lot of people will, for some reason, see the two calls to f and jump to $O(N^2)$. This is completely incorrect.

Rather than making assumptions, let's derive the runtime by walking through the code. Suppose we call $f(4)$. This calls $f(3)$ twice. Each of those calls to $f(3)$ calls $f(2)$, until we get down to $f(1)$.



How many calls are in this tree? (Don't count!)

The tree will have depth N . Each node (i.e., function call) has two children. Therefore, each level will have twice as many calls as the one above it. The number of nodes on each level is:

Level	# Nodes	Also expressed as...	Or...
0	1		2^0
1	2	$2 * \text{previous level} = 2$	2^1
2	4	$2 * \text{previous level} = 2 * 2^1 = 2^2$	2^2
3	8	$2 * \text{previous level} = 2 * 2^2 = 2^3$	2^3
4	16	$2 * \text{previous level} = 2 * 2^3 = 2^4$	2^4

Therefore, there will be $2^0 + 2^1 + 2^2 + 2^3 + 2^4 + \dots + 2^N$ (which is $2^{N+1} - 1$) nodes. (See "Sum of Powers of 2" on page 630.)

Try to remember this pattern. When you have a recursive function that makes multiple calls, the runtime will often (but not always) look like $O(\text{branches}^{\text{depth}})$, where branches is the number of times each recursive call branches. In this case, this gives us $O(2^N)$.

As you may recall, the base of a log doesn't matter for big O since logs of different bases are only different by a constant factor. However, this does not apply to exponents. The base of an exponent does matter. Compare 2^n and 8^n . If you expand 8^n , you get $(2^3)^n$, which equals 2^{3n} , which equals $2^{2n} * 2^n$. As you can see, 8^n and 2^n are different by a factor of 2^{2n} . That is very much not a constant factor!

The space complexity of this algorithm will be $O(N)$. Although we have $O(2^N)$ nodes in the tree total, only $O(N)$ exist at any given time. Therefore, we would only need to have $O(N)$ memory available.

► Examples and Exercises

Big O time is a difficult concept at first. However, once it "clicks," it gets fairly easy. The same patterns come up again and again, and the rest you can derive.

We'll start off easy and get progressively more difficult.

Example 1

What is the runtime of the below code?

```

1 void foo(int[] array) {
2     int sum = 0;
3     int product = 1;
4     for (int i = 0; i < array.length; i++) {
5         sum += array[i];
6     }
7     for (int i = 0; i < array.length; i++) {
8         product *= array[i];
9     }
10    System.out.println(sum + ", " + product);
11 }

```

This will take $O(N)$ time. The fact that we iterate through the array twice doesn't matter.

Example 2

What is the runtime of the below code?

```

1 void printPairs(int[] array) {
2     for (int i = 0; i < array.length; i++) {
3         for (int j = 0; j < array.length; j++) {
4             System.out.println(array[i] + "," + array[j]);
5         }
6     }
7 }

```

The inner for loop has $O(N)$ iterations and it is called N times. Therefore, the runtime is $O(N^2)$.

Another way we can see this is by inspecting what the "meaning" of the code is. It is printing all pairs (two-element sequences). There are $O(N^2)$ pairs; therefore, the runtime is $O(N^2)$.

Example 3

This is very similar code to the above example, but now the inner for loop starts at $i + 1$.

```

1 void printUnorderedPairs(int[] array) {
2     for (int i = 0; i < array.length; i++) {
3         for (int j = i + 1; j < array.length; j++) {
4             System.out.println(array[i] + "," + array[j]);
5         }
6     }
7 }

```

We can derive the runtime several ways.

This pattern of for loop is very common. It's important that you know the runtime and that you deeply understand it. You can't rely on just memorizing common runtimes. Deep comprehension is important.

Counting the Iterations

The first time through j runs for $N-1$ steps. The second time, it's $N-2$ steps. Then $N-3$ steps. And so on.

Therefore, the number of steps total is:

$$(N-1) + (N-2) + (N-3) + \dots + 2 + 1$$

$$= 1 + 2 + 3 + \dots + N-1$$

$$= \text{sum of 1 through } N-1$$

The sum of 1 through $N-1$ is $\frac{N(N-1)}{2}$ (see “Sum of Integers 1 through N ” on page 630), so the runtime will be $O(N^2)$.

What It Means

Alternatively, we can figure out the runtime by thinking about what the code “means.” It iterates through each pair of values for (i, j) where j is bigger than i .

There are N^2 total pairs. Roughly half of those will have $i < j$ and the remaining half will have $i > j$. This code goes through roughly $N^2/2$ pairs so it does $O(N^2)$ work.

Visualizing What It Does

The code iterates through the following (i, j) pairs when $N = 8$:

```
(0, 1) (0, 2) (0, 3) (0, 4) (0, 5) (0, 6) (0, 7)
      (1, 2) (1, 3) (1, 4) (1, 5) (1, 6) (1, 7)
          (2, 3) (2, 4) (2, 5) (2, 6) (2, 7)
              (3, 4) (3, 5) (3, 6) (3, 7)
                  (4, 5) (4, 6) (4, 7)
                      (5, 6) (5, 7)
                          (6, 7)
```

This looks like half of an $N \times N$ matrix, which has size (roughly) $N^2/2$. Therefore, it takes $O(N^2)$ time.

Average Work

We know that the outer loop runs N times. How much work does the inner loop do? It varies across iterations, but we can think about the average iteration.

What is the average value of 1, 2, 3, 4, 5, 6, 7, 8, 9, 10? The average value will be in the middle, so it will be *roughly* 5. (We could give a more precise answer, of course, but we don’t need to for big O .)

What about for 1, 2, 3, ..., N ? The average value in this sequence is $N/2$.

Therefore, since the inner loop does $N/2$ work on average and it is run N times, the total work is $N^2/2$ which is $O(N^2)$.

Example 4

This is similar to the above, but now we have two different arrays.

```
1 void printUnorderedPairs(int[] arrayA, int[] arrayB) {
2     for (int i = 0; i < arrayA.length; i++) {
3         for (int j = 0; j < arrayB.length; j++) {
4             if (arrayA[i] < arrayB[j]) {
5                 System.out.println(arrayA[i] + "," + arrayB[j]);
6             }
7         }
8     }
9 }
```

We can break up this analysis. The if-statement within j ’s for loop is $O(1)$ time since it’s just a sequence of constant-time statements.

We now have this:

```
1 void printUnorderedPairs(int[] arrayA, int[] arrayB) {
```

```

2    for (int i = 0; i < arrayA.length; i++) {
3        for (int j = 0; j < arrayB.length; j++) {
4            /* O(1) work */
5        }
6    }
7 }

```

For each element of arrayA, the inner for loop goes through b iterations, where $b = \text{arrayB.length}$. If $a = \text{arrayA.length}$, then the runtime is $O(ab)$.

If you said $O(N^2)$, then remember your mistake for the future. It's not $O(N^2)$ because there are two different inputs. Both matter. This is an extremely common mistake.

Example 5

What about this strange bit of code?

```

1 void printUnorderedPairs(int[] arrayA, int[] arrayB) {
2     for (int i = 0; i < arrayA.length; i++) {
3         for (int j = 0; j < arrayB.length; j++) {
4             for (int k = 0; k < 100000; k++) {
5                 System.out.println(arrayA[i] + "," + arrayB[j]);
6             }
7         }
8     }
9 }

```

Nothing has really changed here. 100,000 units of work is still constant, so the runtime is $O(ab)$.

Example 6

The following code reverses an array. What is its runtime?

```

1 void reverse(int[] array) {
2     for (int i = 0; i < array.length / 2; i++) {
3         int other = array.length - i - 1;
4         int temp = array[i];
5         array[i] = array[other];
6         array[other] = temp;
7     }
8 }

```

This algorithm runs in $O(N)$ time. The fact that it only goes through half of the array (in terms of iterations) does not impact the big O time.

Example 7

Which of the following are equivalent to $O(N)$? Why?

- $O(N + P)$, where $P < \frac{N}{2}$
- $O(2N)$
- $O(N + \log N)$
- $O(N + M)$

Let's go through these.

- If $P < \frac{N}{2}$, then we know that N is the dominant term so we can drop the $O(P)$.
- $O(2N)$ is $O(N)$ since we drop constants.

- $O(N)$ dominates $O(\log N)$, so we can drop the $O(\log N)$.
- There is no established relationship between N and M , so we have to keep both variables in there.

Therefore, all but the last one are equivalent to $O(N)$.

Example 8

Suppose we had an algorithm that took in an array of strings, sorted each string, and then sorted the full array. What would the runtime be?

Many candidates will reason the following: sorting each string is $O(N \log N)$ and we have to do this for each string, so that's $O(N * N \log N)$. We also have to sort this array, so that's an additional $O(N \log N)$ work. Therefore, the total runtime is $O(N^2 \log N + N \log N)$, which is just $O(N^2 \log N)$.

This is completely incorrect. Did you catch the error?

The problem is that we used N in two different ways. In one case, it's the length of the string (which string?). And in another case, it's the length of the array.

In your interviews, you can prevent this error by either not using the variable "N" at all, or by only using it when there is no ambiguity as to what N could represent.

In fact, I wouldn't even use a and b here, or m and n . It's too easy to forget which is which and mix them up. An $O(a^2)$ runtime is completely different from an $O(a*b)$ runtime.

Let's define new terms—and use names that are logical.

- Let s be the length of the longest string.
- Let a be the length of the array.

Now we can work through this in parts:

- Sorting each string is $O(s \log s)$.
- We have to do this for every string (and there are a strings), so that's $O(a*s \log s)$.
- Now we have to sort all the strings. There are a strings, so you'll may be inclined to say that this takes $O(a \log a)$ time. This is what most candidates would say. You should also take into account that you need to compare the strings. Each string comparison takes $O(s)$ time. There are $O(a \log a)$ comparisons, therefore this will take $O(a*s \log a)$ time.

If you add up these two parts, you get $O(a*s(\log a + \log s))$.

This is it. There is no way to reduce it further.

Example 9

The following simple code sums the values of all the nodes in a balanced binary search tree. What is its runtime?

```
1 int sum(Node node) {
2     if (node == null) {
3         return 0;
4     }
5     return sum(node.left) + node.value + sum(node.right);
6 }
```

Just because it's a binary search tree doesn't mean that there is a log in it!

We can look at this two ways.

What It Means

The most straightforward way is to think about what this means. This code touches each node in the tree once and does a constant time amount of work with each “touch” (excluding the recursive calls).

Therefore, the runtime will be linear in terms of the number of nodes. If there are N nodes, then the runtime is $O(N)$.

Recursive Pattern

On page 44, we discussed a pattern for the runtime of recursive functions that have multiple branches. Let’s try that approach here.

We said that the runtime of a recursive function with multiple branches is typically $O(\text{branches}^{\text{depth}})$. There are two branches at each call, so we’re looking at $O(2^{\text{depth}})$.

At this point many people might assume that something went wrong since we have an exponential algorithm—that something in our logic is flawed or that we’ve inadvertently created an exponential time algorithm (yikes!).

The second statement is correct. We do have an exponential time algorithm, but it’s not as bad as one might think. Consider what variable it’s exponential with respect to.

What is depth? The tree is a balanced binary search tree. Therefore, if there are N total nodes, then depth is roughly $\log N$.

By the equation above, we get $O(2^{\log N})$.

Recall what $\log_2$ means:

$$2^P = Q \rightarrow \log_2 Q = P$$

What is $2^{\log N}$? There is a relationship between 2 and log, so we should be able to simplify this.

Let $P = 2^{\log N}$. By the definition of $\log_2$, we can write this as $\log_2 P = \log_2 N$. This means that $P = N$.

$$\begin{aligned} \text{Let } P &= 2^{\log N} \\ \rightarrow \log_2 P &= \log_2 N \\ \rightarrow P &= N \\ \rightarrow 2^{\log N} &= N \end{aligned}$$

Therefore, the runtime of this code is $O(N)$, where N is the number of nodes.

Example 10

The following method checks if a number is prime by checking for divisibility on numbers less than it. It only needs to go up to the square root of n because if n is divisible by a number greater than its square root then it’s divisible by something smaller than it.

For example, while 33 is divisible by 11 (which is greater than the square root of 33), the “counterpart” to 11 is 3 ($3 * 11 = 33$). 33 will have already been eliminated as a prime number by 3.

What is the time complexity of this function?

```
1  boolean isPrime(int n) {
2      for (int x = 2; x * x <= n; x++) {
3          if (n % x == 0) {
4              return false;
5          }
6      }
7      return true;
}
```

```
8 }
```

Many people get this question wrong. If you're careful about your logic, it's fairly easy.

The work inside the for loop is constant. Therefore, we just need to know how many iterations the for loop goes through in the worst case.

The for loop will start when $x = 2$ and end when $x * x = n$. Or, in other words, it stops when $x = \sqrt{n}$ (when x equals the square root of n).

This for loop is really something like this:

```
1 boolean isPrime(int n) {
2     for (int x = 2; x <= sqrt(n); x++) {
3         if (n % x == 0) {
4             return false;
5         }
6     }
7     return true;
8 }
```

This runs in $O(\sqrt{n})$ time.

Example 11

The following code computes $n!$ (n factorial). What is its time complexity?

```
1 int factorial(int n) {
2     if (n < 0) {
3         return -1;
4     } else if (n == 0) {
5         return 1;
6     } else {
7         return n * factorial(n - 1);
8     }
9 }
```

This is just a straight recursion from n to $n-1$ to $n-2$ down to 1. It will take $O(n)$ time.

Example 12

This code counts all permutations of a string.

```
1 void permutation(String str) {
2     permutation(str, "");
3 }
4
5 void permutation(String str, String prefix) {
6     if (str.length() == 0) {
7         System.out.println(prefix);
8     } else {
9         for (int i = 0; i < str.length(); i++) {
10             String rem = str.substring(0, i) + str.substring(i + 1);
11             permutation(rem, prefix + str.charAt(i));
12         }
13     }
14 }
```

This is a (very!) tricky one. We can think about this by looking at how many times permutation gets called and how long each call takes. We'll aim for getting as tight of an upper bound as possible.

How many times does permutation get called in its base case?

If we were to generate a permutation, then we would need to pick characters for each “slot.” Suppose we had 7 characters in the string. In the first slot, we have 7 choices. Once we pick the letter there, we have 6 choices for the next slot. (Note that this is 6 choices *for each* of the 7 choices earlier.) Then 5 choices for the next slot, and so on.

Therefore, the total number of options is $7 * 6 * 5 * 4 * 3 * 2 * 1$, which is also expressed as $7!$ (7 factorial).

This tells us that there are $n!$ permutations. Therefore, `permutation` is called $n!$ times in its base case (when `prefix` is the full permutation).

How many times does permutation get called before its base case?

But, of course, we also need to consider how many times lines 9 through 12 are hit. Picture a large call tree representing all the calls. There are $n!$ leaves, as shown above. Each leaf is attached to a path of length n . Therefore, we know there will be no more than $n * n!$ nodes (function calls) in this tree.

How long does each function call take?

Executing line 7 takes $O(n)$ time since each character needs to be printed.

Line 10 and line 11 will also take $O(n)$ time combined, due to the string concatenation. Observe that the sum of the lengths of `rem`, `prefix`, and `str.charAt(i)` will always be n .

Each node in our call tree therefore corresponds to $O(n)$ work.

What is the total runtime?

Since we are calling `permutation` $O(n * n!)$ times (as an upper bound), and each one takes $O(n)$ time, the total runtime will not exceed $O(n^2 * n!)$.

Through more complex mathematics, we can derive a tighter runtime equation (though not necessarily a nice closed-form expression). This would almost certainly be beyond the scope of any normal interview.

Example 13

The following code computes the N th Fibonacci number.

```
1 int fib(int n) {
2     if (n <= 0) return 0;
3     else if (n == 1) return 1;
4     return fib(n - 1) + fib(n - 2);
5 }
```

We can use the earlier pattern we’d established for recursive calls: $O(\text{branches}^{\text{depth}})$.

There are 2 branches per call, and we go as deep as N , therefore the runtime is $O(2^N)$.

Through some very complicated math, we can actually get a tighter runtime. The time is indeed exponential, but it’s actually closer to $O(1.6^N)$. The reason that it’s not exactly $O(2^N)$ is that, at the bottom of the call stack, there is sometimes only one call. It turns out that a lot of the nodes are at the bottom (as is true in most trees), so this single versus double call actually makes a big difference. Saying $O(2^N)$ would suffice for the scope of an interview, though (and is still technically correct, if you read the note about big theta on page 39). You might get “bonus points” if you can recognize that it’ll actually be less than that.

Generally speaking, when you see an algorithm with multiple recursive calls, you're looking at exponential runtime.

Example 14

The following code prints all Fibonacci numbers from 0 to n. What is its time complexity?

```
1 void allFib(int n) {
2     for (int i = 0; i < n; i++) {
3         System.out.println(i + ": " + fib(i));
4     }
5 }
6
7 int fib(int n) {
8     if (n <= 0) return 0;
9     else if (n == 1) return 1;
10    return fib(n - 1) + fib(n - 2);
11 }
```

Many people will rush to concluding that since $\text{fib}(n)$ takes $O(2^n)$ time and it's called n times, then it's $O(n2^n)$.

Not so fast. Can you find the error in the logic?

The error is that the n is changing. Yes, $\text{fib}(n)$ takes $O(2^n)$ time, but it matters what that value of n is.

Instead, let's walk through each call.

```
fib(1) -> 21 steps
fib(2) -> 22 steps
fib(3) -> 23 steps
fib(4) -> 24 steps
...
fib(n) -> 2n steps
```

Therefore, the total amount of work is:

$$2^1 + 2^2 + 2^3 + 2^4 + \dots + 2^n$$

As we showed on page 44, this is 2^{n+1} . Therefore, the runtime to compute the first n Fibonacci numbers (using this terrible algorithm) is still $O(2^n)$.

Example 15

The following code prints all Fibonacci numbers from 0 to n. However, this time, it stores (i.e., caches) previously computed values in an integer array. If it has already been computed, it just returns the cache. What is its runtime?

```
1 void allFib(int n) {
2     int[] memo = new int[n + 1];
3     for (int i = 0; i < n; i++) {
4         System.out.println(i + ": " + fib(i, memo));
5     }
6 }
7
8 int fib(int n, int[] memo) {
9     if (n <= 0) return 0;
10    else if (n == 1) return 1;
11    else if (memo[n] > 0) return memo[n];
12
13    memo[n] = fib(n - 1, memo) + fib(n - 2, memo);
```

```

14     return memo[n];
15 }

```

Let's walk through what this algorithm does.

```

fib(1) -> return 1
fib(2)
    fib(1) -> return 1
    fib(0) -> return 0
    store 1 at memo[2]
fib(3)
    fib(2) -> lookup memo[2] -> return 1
    fib(1) -> return 1
    store 2 at memo[3]
fib(4)
    fib(3) -> lookup memo[3] -> return 2
    fib(2) -> lookup memo[2] -> return 1
    store 3 at memo[4]
fib(5)
    fib(4) -> lookup memo[4] -> return 3
    fib(3) -> lookup memo[3] -> return 2
    store 5 at memo[5]
...

```

At each call to `fib(i)`, we have already computed and stored the values for `fib(i-1)` and `fib(i-2)`. We just look up those values, sum them, store the new result, and return. This takes a constant amount of time.

We're doing a constant amount of work N times, so this is $O(n)$ time.

This technique, called memoization, is a very common one to optimize exponential time recursive algorithms.

Example 16

The following function prints the powers of 2 from 1 through n (inclusive). For example, if n is 4, it would print 1, 2, and 4. What is its runtime?

```

1  int powersOf2(int n) {
2      if (n < 1) {
3          return 0;
4      } else if (n == 1) {
5          System.out.println(1);
6          return 1;
7      } else {
8          int prev = powersOf2(n / 2);
9          int curr = prev * 2;
10         System.out.println(curr);
11         return curr;
12     }
13 }

```

There are several ways we could compute this runtime.

What It Does

Let's walk through a call like `powersOf2(50)`.

```

powersOf2(50)
-> powersOf2(25)

```



```

-> powersOf2(12)
  -> powersOf2(6)
    -> powersOf2(3)
      -> powersOf2(1)
        -> print & return 1
      print & return 2
    print & return 4
  print & return 8
print & return 16
print & return 32

```

The runtime, then, is the number of times we can divide 50 (or n) by 2 until we get down to the base case (1). As we discussed on page 44, the number of times we can halve n until we get 1 is $O(\log n)$.

What It Means

We can also approach the runtime by thinking about what the code is supposed to be doing. It's supposed to be computing the powers of 2 from 1 through n .

Each call to `powersOf2` results in exactly one number being printed and returned (excluding what happens in the recursive calls). So if the algorithm prints 13 values at the end, then `powersOf2` was called 13 times.

In this case, we are told that it prints all the powers of 2 between 1 and n . Therefore, the number of times the function is called (which will be its runtime) must equal the number of powers of 2 between 1 and n .

There are $\log N$ powers of 2 between 1 and n . Therefore, the runtime is $O(\log n)$.

Rate of Increase

A final way to approach the runtime is to think about how the runtime changes as n gets bigger. After all, this is exactly what big O time means.

If N goes from P to $P+1$, the number of calls to `powersOfTwo` might not change at all. When will the number of calls to `powersOfTwo` increase? It will increase by 1 each time n doubles in size.

So, each time n doubles, the number of calls to `powersOfTwo` increases by 1. Therefore, the number of calls to `powersOfTwo` is the number of times you can double 1 until you get n . It is x in the equation $2^x = n$.

What is x ? The value of x is $\log n$. This is exactly what meant by $x = \log n$.

Therefore, the runtime is $O(\log n)$.

Additional Problems

VI.1 The following code computes the product of a and b . What is its runtime?

```

int product(int a, int b) {
    int sum = 0;
    for (int i = 0; i < b; i++) {
        sum += a;
    }
    return sum;
}

```

VI.2 The following code computes a^b . What is its runtime?

```

int power(int a, int b) {
    if (b < 0) {

```

```

        return 0; // error
    } else if (b == 0) {
        return 1;
    } else {
        return a * power(a, b - 1);
    }
}

```

VI.3 The following code computes $a \% b$. What is its runtime?

```

int mod(int a, int b) {
    if (b <= 0) {
        return -1;
    }
    int div = a / b;
    return a - div * b;
}

```

VI.4 The following code performs integer division. What is its runtime (assume a and b are both positive)?

```

int div(int a, int b) {
    int count = 0;
    int sum = b;
    while (sum <= a) {
        sum += b;
        count++;
    }
    return count;
}

```

VI.5 The following code computes the [integer] square root of a number. If the number is not a perfect square (there is no integer square root), then it returns -1. It does this by successive guessing. If n is 100, it first guesses 50. Too high? Try something lower – halfway between 1 and 50. What is its runtime?

```

int sqrt(int n) {
    return sqrt_helper(n, 1, n);
}

int sqrt_helper(int n, int min, int max) {
    if (max < min) return -1; // no square root

    int guess = (min + max) / 2;
    if (guess * guess == n) { // found it!
        return guess;
    } else if (guess * guess < n) { // too low
        return sqrt_helper(n, guess + 1, max); // try higher
    } else { // too high
        return sqrt_helper(n, min, guess - 1); // try lower
    }
}

```

VI.6 The following code computes the [integer] square root of a number. If the number is not a perfect square (there is no integer square root), then it returns -1. It does this by trying increasingly large numbers until it finds the right value (or is too high). What is its runtime?

```

int sqrt(int n) {
    for (int guess = 1; guess * guess <= n; guess++) {
        if (guess * guess == n) {
            return guess;
        }
    }
}

```

```

    }
}
return -1;
}

```

VI.7 If a binary search tree is not balanced, how long might it take (worst case) to find an element in it?

VI.8 You are looking for a specific value in a binary tree, but the tree is not a binary search tree. What is the time complexity of this?

VI.9 The `appendToNew` method appends a value to an array by creating a new, longer array and returning this longer array. You've used the `appendToNew` method to create a `copyArray` function that repeatedly calls `appendToNew`. How long does copying an array take?

```

int[] copyArray(int[] array) {
    int[] copy = new int[0];
    for (int value : array) {
        copy = appendToNew(copy, value);
    }
    return copy;
}

```

```

int[] appendToNew(int[] array, int value) {
    // copy all elements over to new array
    int[] bigger = new int[array.length + 1];
    for (int i = 0; i < array.length; i++) {
        bigger[i] = array[i];
    }

    // add new element
    bigger[bigger.length - 1] = value;
    return bigger;
}

```

VI.10 The following code sums the digits in a number. What is its big O time?

```

int sumDigits(int n) {
    int sum = 0;
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}

```

VI.11 The following code prints all strings of length `k` where the characters are in sorted order. It does this by generating all strings of length `k` and then checking if each is sorted. What is its runtime?

```

int numChars = 26;

void printSortedStrings(int remaining) {
    printSortedStrings(remaining, "");
}

void printSortedStrings(int remaining, String prefix) {
    if (remaining == 0) {
        if (isInOrder(prefix)) {
            System.out.println(prefix);
        }
    }
}

```

```

    } else {
        for (int i = 0; i < numChars; i++) {
            char c = ithLetter(i);
            printSortedStrings(remaining - 1, prefix + c);
        }
    }
}

boolean isInOrder(String s) {
    for (int i = 1; i < s.length(); i++) {
        int prev = ithLetter(s.charAt(i - 1));
        int curr = ithLetter(s.charAt(i));
        if (prev > curr) {
            return false;
        }
    }
    return true;
}

char ithLetter(int i) {
    return (char) (((int) 'a') + i);
}

```

VI.12 The following code computes the intersection (the number of elements in common) of two arrays. It assumes that neither array has duplicates. It computes the intersection by sorting one array (array b) and then iterating through array a checking (via binary search) if each value is in b. What is its runtime?

```

int intersection(int[] a, int[] b) {
    mergesort(b);
    int intersect = 0;

    for (int x : a) {
        if (binarySearch(b, x) >= 0) {
            intersect++;
        }
    }

    return intersect;
}

```

Solutions

1. $O(b)$. The for loop just iterates through b.
2. $O(b)$. The recursive code iterates through b calls, since it subtracts one at each level.
3. $O(1)$. It does a constant amount of work.
4. $O(\frac{a}{b})$. The variable count will eventually equal $\frac{a}{b}$. The while loop iterates count times. Therefore, it iterates $\frac{a}{b}$ times.
5. $O(\log n)$. This algorithm is essentially doing a binary search to find the square root. Therefore, the runtime is $O(\log n)$.
6. $O(\sqrt{n})$. This is just a straightforward loop that stops when $\text{guess} * \text{guess} > n$ (or, in other words, when $\text{guess} > \sqrt{n}$).

7. $O(n)$, where n is the number of nodes in the tree. The max time to find an element is the depth tree. The tree could be a straight list downwards and have depth n .
8. $O(n)$. Without any ordering property on the nodes, we might have to search through all the nodes.
9. $O(n^2)$, where n is the number of elements in the array. The first call to `appendToNew` takes 1 copy. The second call takes 2 copies. The third call takes 3 copies. And so on. The total time will be the sum of 1 through n , which is $O(n^2)$.
10. $O(\log n)$. The runtime will be the number of digits in the number. A number with d digits can have a value up to 10^d . If $n = 10^d$, then $d = \log n$. Therefore, the runtime is $O(\log n)$.
11. $O(kc^k)$, where k is the length of the string and c is the number of characters in the alphabet. It takes $O(c^k)$ time to generate each string. Then, we need to check that each of these is sorted, which takes $O(k)$ time.
12. $O(b \log b + a \log b)$. First, we have to sort array b , which takes $O(b \log b)$ time. Then, for each element in a , we do binary search in $O(\log b)$ time. The second part takes $O(a \log b)$ time.

VII

Technical Questions

Technical questions form the basis for how many of the top tech companies interview. Many candidates are intimidated by the difficulty of these questions, but there are logical ways to approach them.

► How to Prepare

Many candidates just read through problems and solutions. That's like trying to learn calculus by reading a problem and its answer. You need to practice solving problems. Memorizing solutions won't help you much.

For each problem in this book (and any other problem you might encounter), do the following:

1. *Try to solve the problem on your own.* Hints are provided at the back of this book, but push yourself to develop a solution with as little help as possible. Many questions are designed to be tough—that's okay! When you're solving a problem, make sure to think about the space and time efficiency.
2. *Write the code on paper.* Coding on a computer offers luxuries such as syntax highlighting, code completion, and quick debugging. Coding on paper does not. Get used to this—and to how slow it is to write and edit code—by coding on paper.
3. *Test your code—on paper.* This means testing the general cases, base cases, error cases, and so on. You'll need to do this during your interview, so it's best to practice this in advance.
4. *Type your paper code as-is into a computer.* You will probably make a bunch of mistakes. Start a list of all the errors you make so that you can keep these in mind during the actual interview.

In addition, try to do as many mock interviews as possible. You and a friend can take turns giving each other mock interviews. Though your friend may not be an expert interviewer, he or she may still be able to walk you through a coding or algorithm problem. You'll also learn a lot by experiencing what it's like to be an interviewer.

► What You Need To Know

The sorts of data structure and algorithm questions that many companies focus on are not knowledge tests. However, they do assume a baseline of knowledge.

Core Data Structures, Algorithms, and Concepts

Most interviewers won't ask about specific algorithms for binary tree balancing or other complex algorithms. Frankly, being several years out of school, they probably don't remember these algorithms either.

You're usually only expected to know the basics. Here's a list of the absolute, must-have knowledge:

Data Structures	Algorithms	Concepts
Linked Lists	Breadth-First Search	Bit Manipulation
Trees, Tries, & Graphs	Depth-First Search	Memory (Stack vs. Heap)
Stacks & Queues	Binary Search	Recursion
Heaps	Merge Sort	Dynamic Programming
Vectors / ArrayLists	Quick Sort	Big O Time & Space
Hash Tables		

For each of these topics, make sure you understand how to use and implement them and, where applicable, the space and time complexity.

Practicing implementing the data structures and algorithm (on paper, and then on a computer) is also a great exercise. It will help you learn how the internals of the data structures work, which is important for many interviews.

Did you miss that paragraph above? It's important. If you don't feel very, very comfortable with each of the data structures and algorithms listed, practice implementing them from scratch.

In particular, hash tables are an extremely important topic. Make sure you are very comfortable with this data structure.

Powers of 2 Table

The table below is useful for many questions involving scalability or any sort of memory limitation. Memorizing this table isn't strictly required, but it can be useful. You should at least be comfortable deriving it.

Power of 2	Exact Value (X)	Approx. Value	X Bytes into MB, GB, etc.
7	128		
8	256		
10	1024	1 thousand	1 KB
16	65,536		64 KB
20	1,048,576	1 million	1 MB
30	1,073,741,824	1 billion	1 GB
32	4,294,967,296		4 GB
40	1,099,511,627,776	1 trillion	1 TB

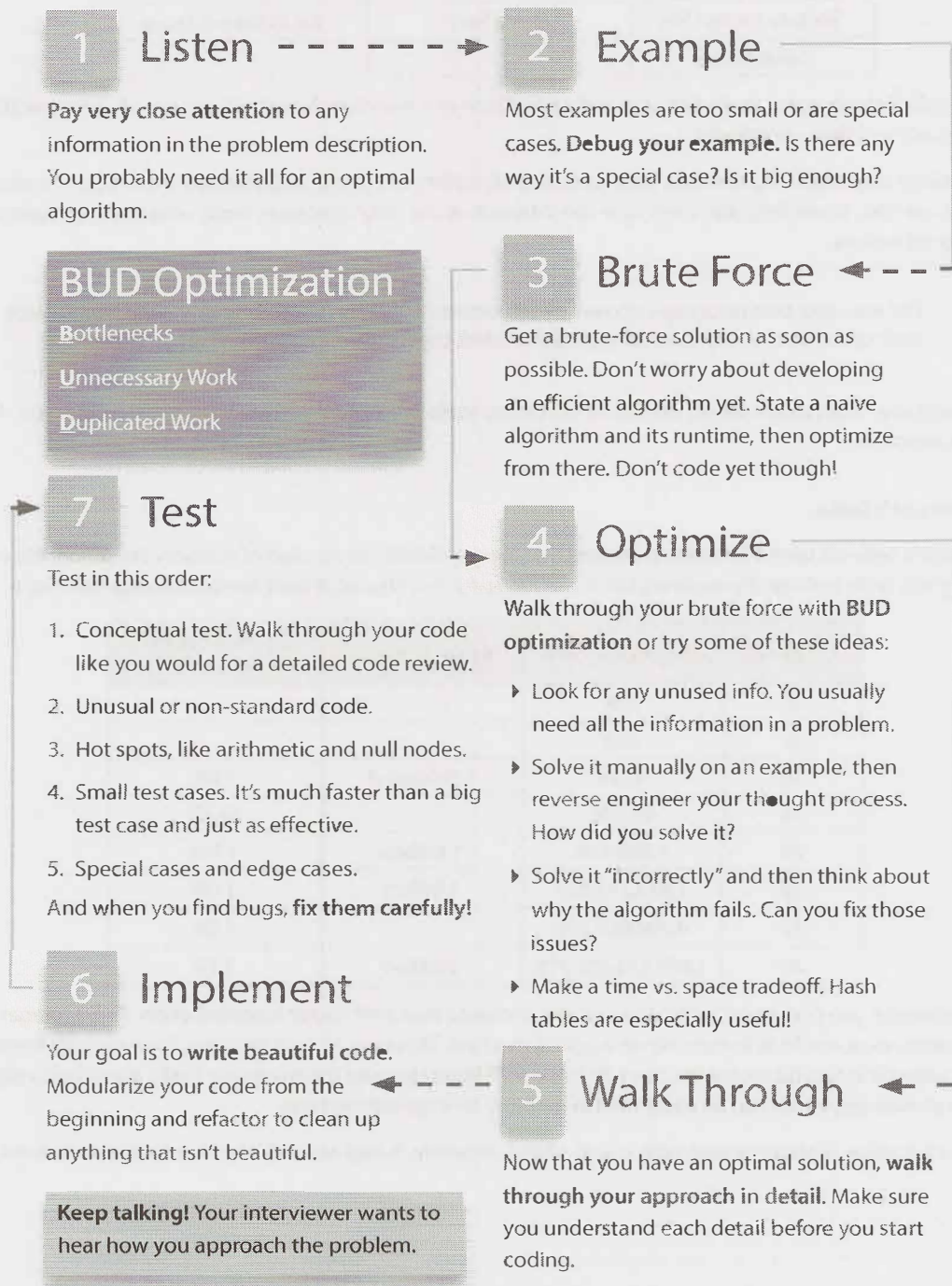
For example, you could use this table to quickly compute that a bit vector mapping every 32-bit integer to a boolean value could fit in memory on a typical machine. There are 2^{32} such integers. Because each integer takes one bit in this bit vector, we need 2^{32} bits (or 2^{29} bytes) to store this mapping. That's about half a gigabyte of memory, which can be easily held in memory on a typical machine.

If you are doing a phone screen with a web-based company, it may be useful to have this table in front of you.

► Walking Through a Problem

The below map/flowchart walks you through how to solve a problem. Use this in your practice. You can download this handout and more at CrackingTheCodingInterview.com.

A Problem-Solving Flowchart



We'll go through this flowchart in more detail.

What to Expect

Interviews are supposed to be difficult. If you don't get every—or any—answer immediately, that's okay! That's the normal experience, and it's not bad.

Listen for guidance from the interviewer. The interviewer might take a more active or less active role in your problem solving. The level of interviewer participation depends on your performance, the difficulty of the question, what the interviewer is looking for, and the interviewer's own personality.

When you're given a problem (or when you're practicing), work your way through it using the approach below.

1. Listen Carefully

You've likely heard this advice before, but I'm saying something a bit more than the standard "make sure you hear the problem correctly" advice.

Yes, you do want to listen to the problem and make sure you heard it correctly. You do want to ask questions about anything you're unsure about.

But I'm saying something more than that.

Listen carefully to the problem, and be sure that you've mentally recorded any *unique* information in the problem.

For example, suppose a question starts with one of the following lines. It's reasonable to assume that the information is there for a reason.

- "Given two arrays that are sorted, find ..."
You probably need to know that the data is sorted. The optimal algorithm for the sorted situation is probably different than the optimal algorithm for the unsorted situation.
- "Design an algorithm to be run repeatedly on a server that ..."
The server/to-be-run-repeatedly situation is different from the run-once situation. Perhaps this means that you cache data? Or perhaps it justifies some reasonable precomputation on the initial dataset?

It's unlikely (although not impossible) that your interviewer would give you this information if it didn't affect the algorithm.

Many candidates will hear the problem correctly. But ten minutes into developing an algorithm, some of the key details of the problem have been forgotten. Now they are in a situation where they actually can't solve the problem optimally.

Your first algorithm doesn't need to use the information. But if you find yourself stuck, or you're still working to develop something more optimal, ask yourself if you've used all the information in the problem.

You might even find it useful to write the pertinent information on the whiteboard.

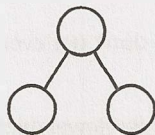
2. Draw an Example

An example can dramatically improve your ability to solve an interview question, and yet so many candidates just try to solve the question in their heads.

When you hear a question, get out of your chair, go to the whiteboard, and draw an example.

There's an art to drawing an example though. You want a good example.

Very typically, a candidate might draw something like this for an example of a binary search tree:



This is a bad example for several reasons. First, it's too small. You will have trouble finding a pattern in such a small example. Second, it's not specific. A binary search tree has values. What if the numbers tell you something about how to approach the problem? Third, it's actually a special case. It's not just a balanced tree, but it's also a beautiful, perfect tree where every node other than the leaves has two children. Special cases can be very deceiving.

Instead, you want to create an example that is:

- Specific. It should use real numbers or strings (if applicable to the problem).
- Sufficiently large. Most examples are too small, by about 50%.
- Not a special case. Be careful. It's very easy to inadvertently draw a special case. If there's any way your example is a special case (even if you think it probably won't be a big deal), you should fix it.

Try to make the best example you can. If it later turns out your example isn't quite right, you can and should fix it.

3. State a Brute Force

Once you have an example done (actually, you can switch the order of steps 2 and 3 in some problems), state a brute force. It's okay and expected that your initial algorithm won't be very optimal.

Some candidates don't state the brute force because they think it's both obvious and terrible. But here's the thing: Even if it's obvious for you, it's not necessarily obvious for all candidates. You don't want your interviewer to think that you're struggling to see even the easy solution.

It's okay that this initial solution is terrible. Explain what the space and time complexity is, and then dive into improvements.

Despite being possibly slow, a brute force algorithm is valuable to discuss. It's a starting point for optimizations, and it helps you wrap your head around the problem.

4. Optimize

Once you have a brute force algorithm, you should work on optimizing it. A few techniques that work well are:

1. Look for any unused information. Did your interviewer tell you that the array was sorted? How can you leverage that information?
2. Use a fresh example. Sometimes, just seeing a different example will unclog your mind or help you see a pattern in the problem.
3. Solve it "incorrectly." Just like having an inefficient solution can help you find an efficient solution, having an incorrect solution might help you find a correct solution. For example, if you're asked to generate a

random value from a set such that all values are equally likely, an incorrect solution might be one that returns a semi-random value: Any value could be returned, but some are more likely than others. You can then think about why that solution isn't perfectly random. Can you rebalance the probabilities?

4. Make time vs. space tradeoff. Sometimes storing extra state about the problem can help you optimize the runtime.
5. Precompute information. Is there a way that you can reorganize the data (sorting, etc.) or compute some values upfront that will help save time in the long run?
6. Use a hash table. Hash tables are widely used in interview questions and should be at the top of your mind.
7. Think about the best conceivable runtime (discussed on page 72).

Walk through the brute force with these ideas in mind and look for BUD (page 67).

5. Walk Through

After you've nailed down an optimal algorithm, don't just dive into coding. Take a moment to solidify your understanding of the algorithm.

Whiteboard coding is slow—very slow. So is testing your code and fixing it. As a result, you need to make sure that you get it as close to “perfect” in the beginning as possible.

Walk through your algorithm and get a feel for the structure of the code. Know what the variables are and when they change.

What about pseudocode? You can write pseudocode if you'd like. Be careful about what you write. Basic steps (“(1) Search array. (2) Find biggest. (3) Insert in heap.”) or brief logic (“if $p < q$, move p . else move q ”) can be valuable. But when your pseudocode starts having for loops that are written in plain English, then you're essentially just writing sloppy code. It'd probably be faster to just write the code.

If you don't understand exactly what you're about to write, you'll struggle to code it. It will take you longer to finish the code, and you're more likely to make major errors.

6. Implement

Now that you have an optimal algorithm and you know exactly what you're going to write, go ahead and implement it.

Start coding in the far top left corner of the whiteboard (you'll need the space). Avoid “line creep” (where each line of code is written an awkward slant). It makes your code look messy and can be very confusing when working in a whitespace-sensitive language, like Python.

Remember that you only have a short amount of code to demonstrate that you're a great developer. Everything counts. Write beautiful code.

Beautiful code means:

- Modularized code. This shows good coding style. It also makes things easier for you. If your algorithm uses a matrix initialized to `{{1, 2, 3}, {4, 5, 6}, ...}`, don't waste your time writing this initialization code. Just pretend you have a function `initIncrementalMatrix(int size)`. Fill in the details later if you need to.

- Error checks. Some interviewers care a lot about this, while others don't. A good compromise here is to add a `todo` and then just explain out loud what you'd like to test.
- Use other classes/structs where appropriate. If you need to return a list of start and end points from a function, you could do this as a two-dimensional array. It's better though to do this as a list of `StartEndPair` (or possibly `Range`) objects. You don't necessarily have to fill in the details for the class. Just pretend it exists and deal with the details later if you have time.
- Good variable names. Code that uses single-letter variables everywhere is difficult to read. That's not to say that there's anything wrong with using `i` and `j`, where appropriate (such as in a basic for-loop iterating through an array). However, be careful about where you do this. If you write something like `int i = startOfChild(array)`, there might be a better name for this variable, such as `startChild`.
Long variable names can also be slow to write though. A good compromise that most interviewers will be okay with is to abbreviate it after the first usage. You can use `startChild` the first time, and then explain to your interviewer that you will abbreviate this as `sc` after this.

The specifics of what makes good code vary between interviewers and candidates, and the problem itself. Focus on writing beautiful code, whatever that means to you.

If you see something you can refactor later on, then explain this to your interviewer and decide whether or not it's worth the time to do so. Usually it is, but not always.

If you get confused (which is common), go back to your example and walk through it again.

7. Test

You wouldn't check in code in the real world without testing it, and you shouldn't "submit" code in an interview without testing it either.

There are smart and not-so-smart ways to test your code though.

What many candidates do is take their earlier example and test it against their code. That might discover bugs, but it'll take a really long time to do so. Hand testing is very slow. If you really did use a nice, big example to develop your algorithm, then it'll take you a very long time to find that little off-by-one error at the end of your code.

Instead, try this approach:

1. Start with a "conceptual" test. A conceptual test means just reading and analyzing what each line of code does. Think about it like you're explaining the lines of code for a code reviewer. Does the code do what you think it should do?
2. Weird looking code. Double check that line of code that says `x = length - 2`. Investigate that for loop that starts at `i = 1`. While you undoubtedly did this for a reason, it's really easy to get it just slightly wrong.
3. Hot spots. You've coded long enough to know what things are likely to cause problems. Base cases in recursive code. Integer division. Null nodes in binary trees. The start and end of iteration through a linked list. Double check that stuff.
4. Small test cases. This is the first time we use an actual, specific test case to test the code. Don't use that nice, big 8-element array from the algorithm part. Instead, use a 3 or 4 element array. It'll likely discover the same bugs, but it will be much faster to do so.
5. Special cases. Test your code against null or single element values, the extreme cases, and other special cases.

When you find bugs (and you probably will), you should of course fix them. But don't just make the first correction you think of. Instead, carefully analyze why the bug occurred and ensure that your fix is the best one.

► Optimize & Solve Technique #1: Look for BUD

This is perhaps the most useful approach I've found for optimizing problems. "BUD" is a silly acronym for:

- **B**ottlenecks
- **U**nnecessary work
- **D**uplicated work

These are three of the most common things that an algorithm can "waste" time doing. You can walk through your brute force looking for these things. When you find one of them, you can then focus on getting rid of it.

If it's still not optimal, you can repeat this approach on your current best algorithm.

Bottlenecks

A bottleneck is a part of your algorithm that slows down the overall runtime. There are two common ways this occurs:

- You have one-time work that slows down your algorithm. For example, suppose you have a two-step algorithm where you first sort the array and then you find elements with a particular property. The first step is $O(N \log N)$ and the second step is $O(N)$. Perhaps you could reduce the second step to $O(\log N)$ or $O(1)$, but would it matter? Not too much. It's certainly not a priority, as the $O(N \log N)$ is the bottleneck. Until you optimize the first step, your overall algorithm will be $O(N \log N)$.
- You have a chunk of work that's done repeatedly, like searching. Perhaps you can reduce that from $O(N)$ to $O(\log N)$ or even $O(1)$. That will greatly speed up your overall runtime.

Optimizing a bottleneck can make a big difference in your overall runtime.

Example: Given an array of distinct integer values, count the number of pairs of integers that have difference k . For example, given the array $\{1, 7, 5, 9, 2, 12, 3\}$ and the difference $k = 2$, there are four pairs with difference 2: $(1, 3)$, $(3, 5)$, $(5, 7)$, $(7, 9)$.

A brute force algorithm is to go through the array, starting from the first element, and then search through the remaining elements (which will form the other side of the pair). For each pair, compute the difference. If the difference equals k , increment a counter of the difference.

The bottleneck here is the repeated search for the "other side" of the pair. It's therefore the main thing to focus on optimizing.

How can we more quickly find the right "other side"? Well, we actually know the other side of $(x, ?)$. It's $x + k$ or $x - k$. If we sorted the array, we could find the other side for each of the N elements in $O(\log N)$ time by doing a binary search.

We now have a two-step algorithm, where both steps take $O(N \log N)$ time. Now, sorting is the new bottleneck. Optimizing the second step won't help because the first step is slowing us down anyway.

We just have to get rid of the first step entirely and operate on an unsorted array. How can we find things quickly in an unsorted array? With a hash table.

Throw everything in the array into the hash table. Then, to look up if $x + k$ or $x - k$ exist in the array, we just look it up in the hash table. We can do this in $O(N)$ time.

Unnecessary Work

Example: Print all positive integer solutions to the equation $a^3 + b^3 = c^3 + d^3$ where a, b, c , and d are integers between 1 and 1000.

A brute force solution will just have four nested for loops. Something like:

```
1  n = 1000
2  for a from 1 to n
3    for b from 1 to n
4      for c from 1 to n
5        for d from 1 to n
6          if  $a^3 + b^3 == c^3 + d^3$ 
7            print a, b, c, d
```

This algorithm iterates through all possible values of a, b, c , and d and checks if that combination happens to work.

It's unnecessary to continue checking for other possible values of d . Only one could work. We should at least break after we find a valid solution.

```
1  n = 1000
2  for a from 1 to n
3    for b from 1 to n
4      for c from 1 to n
5        for d from 1 to n
6          if  $a^3 + b^3 == c^3 + d^3$ 
7            print a, b, c, d
8            break // break out of d's loop
```

This won't make a meaningful change to the runtime—our algorithm is still $O(N^4)$ —but it's still a good, quick fix to make.

Is there anything else that is unnecessary? Yes. If there's only one valid d value for each (a, b, c) , then we can just compute it. This is just simple math: $d = \sqrt[3]{a^3 + b^3 - c^3}$.

```
1  n = 1000
2  for a from 1 to n
3    for b from 1 to n
4      for c from 1 to n
5        d = pow( $a^3 + b^3 - c^3$ , 1/3) // Will round to int
6        if  $a^3 + b^3 == c^3 + d^3$  // Validate that the value works
7          print a, b, c, d
```

The `if` statement on line 6 is important. Line 5 will always find a value for d , but we need to check that it's the right integer value.

This will reduce our runtime from $O(N^4)$ to $O(N^3)$.

Duplicated Work

Using the same problem and brute force algorithm as above, let's look for duplicated work this time.

The algorithm operates by essentially iterating through all (a, b) pairs and then searching all (c, d) pairs to find if there are any matches to that (a, b) pair.

Why do we keep on computing all (c, d) pairs for each (a, b) pair? We should just create the list of (c, d) pairs once. Then, when we have an (a, b) pair, find the matches within the (c, d) list. We can quickly locate the matches by inserting each (c, d) pair into a hash table that maps from the sum to the pair (or, rather, the list of pairs that have that sum).

```

1  n = 1000
2  for c from 1 to n
3      for d from 1 to n
4          result = c3 + d3
5          append (c, d) to list at value map[result]
6  for a from 1 to n
7      for b from 1 to n
8          result = a3 + b3
9          list = map.get(result)
10         for each pair in list
11             print a, b, pair

```

Actually, once we have the map of all the (c, d) pairs, we can just use that directly. We don't need to generate the (a, b) pairs. Each (a, b) will already be in the map.

```

1  n = 1000
2  for c from 1 to n
3      for d from 1 to n
4          result = c3 + d3
5          append (c, d) to list at value map[result]
6
7  for each result, list in map
8      for each pair1 in list
9          for each pair2 in list
10             print pair1, pair2

```

This will take our runtime to $O(N^2)$.

► Optimize & Solve Technique #2: DIY (Do It Yourself)

The first time you heard about how to find an element in a sorted array (before being taught binary search), you probably didn't jump to, "Ah ha! We'll compare the target element to the midpoint and then recurse on the appropriate half."

And yet, you could give someone who has no knowledge of computer science an alphabetized pile of student papers and they'll likely implement something like binary search to locate a student's paper. They'll probably say, "Gosh, Peter Smith? He'll be somewhere in the bottom of the stack." They'll pick a random paper in the middle(ish), compare the name to "Peter Smith", and then continue this process on the remainder of the papers. Although they have no knowledge of binary search, they intuitively "get it."

Our brains are funny like this. Throw the phrase "Design an algorithm" in there and people often get all jumbled up. But give people an actual example—whether just of the data (e.g., an array) or of the real-life parallel (e.g., a pile of papers)—and their intuition gives them a very nice algorithm.

I've seen this come up countless times with candidates. Their computer algorithm is extraordinarily slow, but when asked to solve the same problem manually, they immediately do something quite fast. (And it's not too surprisingly, in some sense. Things that are slow for a computer are often slow by hand. Why would you put yourself through extra work?)

Therefore, when you get a question, try just working it through intuitively on a real example. Often a bigger example will be easier.